

TÁC TỬ THÔNG MINH (INTELLIGENT AGENTS)

CHƯƠNG 2

Nhắc nhở

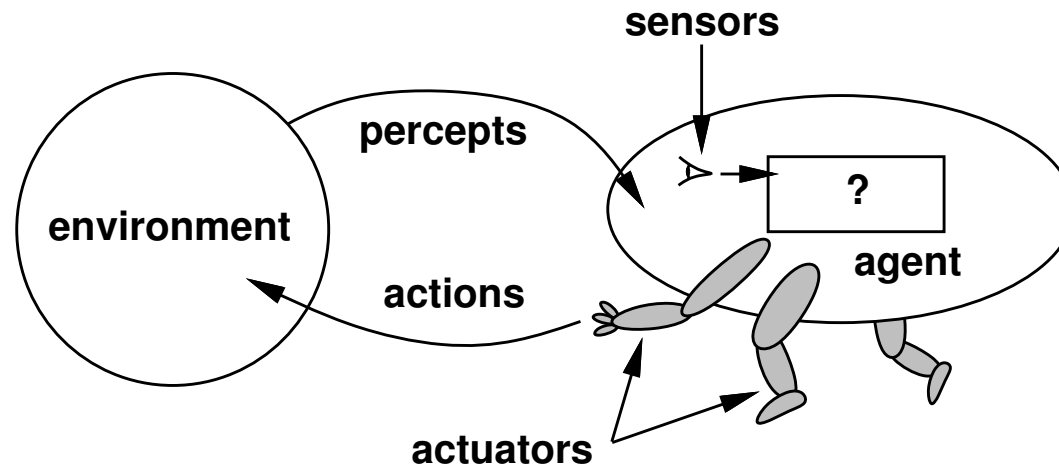
Nhiệm vụ 0 (bồi dưỡng ngọn lửa nói ngọn) hạn chót vào ngày 28/1

Hướng dẫn Lisp/emacs/AIMA: 1-11 hôm nay và thứ Hai, 27/1 Soda

Phác thảo

- ◇ Tác nhân và môi trường
- ◇ Tính hợp lý
- ◇ PEAS (Đo hiệu suất, Môi trường, Bộ truyền động, Cảm biến)
- ◇ Các loại môi trường
- ◇ Các loại tác nhân

Tác nhân và môi trường



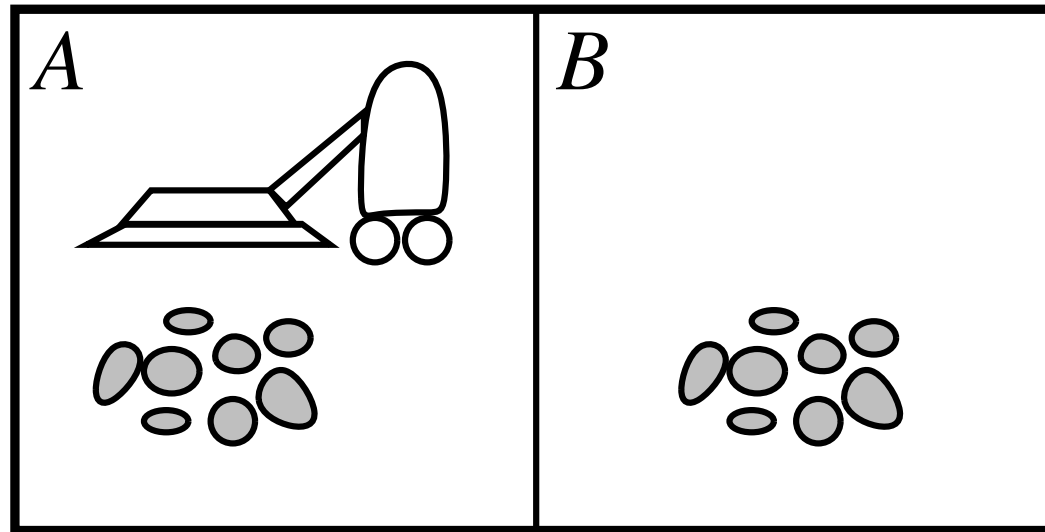
Đại lý bao gồm con người, robot, robot mềm, bộ điều nhiệt, v.v.

Hàm tác nhân ánh xạ từ lịch sử nhận thức đến hành động:

$$f : \mathcal{P}^* \rightarrow \mathcal{A}$$

Chương trình tác nhân chạy trên kiến trúc vật lý để tạo ra f

Thế giới máy hút bụi



Nhận thức: vị trí và nội dung, ví dụ: $[A, Dirty]$

Hành động: *Left*, *Right*, *Suck*, *NoOp*

Chất làm sạch máy hút bụi

Percept sequence	Action
$[A, Clean]$	<i>Right</i>
$[A, Dirty]$	<i>Suck</i>
$[B, Clean]$	<i>Left</i>
$[B, Dirty]$	<i>Suck</i>
$[A, Clean], [A, Clean]$	<i>Right</i>
$[A, Clean], [A, Dirty]$	<i>Suck</i>
$\vdots$	$\vdots$

```

function REFLEX-VACUUM-AGENT(  $[location, status]$ ) returns an action
if  $status = Dirty$  then return Suck
else if  $location = A$  then return Right
else if  $location = B$  then return Left
  
```

Chức năng **right** là gì?

Nó có thể được thực hiện trong một chương trình đại lý nhỏ không?

Tính hợp lý

Đã sửa lỗi đo lường hiệu suất đánh giá trình tự môi trường

- một điểm trên mỗi ô vuông được dọn sạch kịp thời T ?
- một điểm cho mỗi ô vuông sạch trong mỗi bước thời gian, trừ một điểm cho mỗi nước đi?
- phạt cho $> k$ ô vuông bẩn?

Tác nhân hợp lý chọn bất kỳ hành động nào tối đa hóa giá trị **expected** của thước đo hiệu suất **đưa ra trình tự nhận thức cho đến nay**

Hợp lý $\neq$ toàn trí

- nhận thức có thể không cung cấp tất cả thông tin liên quan
- Lý trí $\neq$ thấu thị

– kết quả hành động có thể không như mong đợi
Do đó, hợp lý $\neq$ thành công

Hợp lý $\Rightarrow$ khám phá, học tập, tự chủ

Đậu Hà Lan

Để thiết kế một tác nhân hợp lý, chúng ta phải chỉ định môi trường tác vụ

Ví dụ, hãy xem xét nhiệm vụ thiết kế một chiếc taxi tự động:

Thước đo hiệu suất??

Môi trường??

Bộ truyền động??

Cảm biến??

Đậu Hà Lan

Để thiết kế một tác nhân hợp lý, chúng ta phải chỉ định môi trường tác vụ

Ví dụ, hãy xem xét nhiệm vụ thiết kế một chiếc taxi tự động:

Đo lường hiệu quả hoạt động?? an toàn, điểm đến, lợi nhuận, tính hợp pháp, sự thoải mái, ...

Môi trường?? Đường phố/đường cao tốc ở Hoa Kỳ, giao thông, người đi bộ, thời tiết, ...

Thiết bị truyền động?? tay lái, chân ga, phanh, còi, loa/màn hình, ...

Cảm biến?? video, gia tốc kế, đồng hồ đo, cảm biến động cơ, bàn phím, GPS, ...

Đại lý mua sắm qua Internet

Thước đo hiệu suất??

Môi trường??

Bộ truyền động??

Cảm biến??

Đại lý mua sắm qua Internet

Thước đo hiệu suất?? giá cả, chất lượng, sự phù hợp, hiệu quả

Môi trường?? Các trang web WWW, nhà cung cấp, chủ hàng hiện tại và tương lai

Actuators?? hiển thị cho người dùng, theo dõi URL, điền vào biểu mẫu

Cảm biến?? Trang HTML (văn bản, đồ họa, tập lệnh)

Loại môi trường

	Solitaire Backgammon Mua sắm trên Internet Taxi
<u>Có thể quan sát được??</u> <u>Xác định??</u> <u>Tập??</u> <u>Tĩnh??</u> <u>Rời rạc??</u> <u>Đại lý đơn lẻ??</u>	

Loại môi trường

	Solitaire	Backgammon	Mua sắm trên Internet	Taxi
<u>Có thể quan sát được??</u>	Có	Có	Không	Không
<u>Xác định??</u>				
<u>Tập??</u>				
<u>Tĩnh??</u>				
<u>Rời rạc??</u>				
<u>Đại lý đơn lẻ??</u>				

Loại môi trường

	Solitaire	Backgammon	Mua sắm trên Internet	Taxi
<u>Có thể quan sát được??</u>	Có	Có	Không	Không
<u>Xác định??</u>	Có	Không	Một phần	Không
<u>Tập??</u>				
<u>Tĩnh??</u>				
<u>Rời rạc??</u>				
<u>Đại lý đơn lẻ??</u>				

Loại môi trường

	Solitaire	Backgammon	Mua sắm trên Internet	Taxi
<u>Có thể quan sát được??</u>	Có	Có	Không	Không
<u>Xác định??</u>	Có	Không	Một phần	Không
<u>Tập??</u>	Không	Không	Không	Không
<u>Tĩnh??</u>				
<u>Rời rạc??</u>				
<u>Đại lý đơn lẻ??</u>				

Loại môi trường

	Solitaire	Backgammon	Mua sắm trên Internet	Taxi
<u>Có thể quan sát được??</u>	Có	Có	Không	Không
<u>Xác định??</u>	Có	Không	Một phần	Không
<u>Tập??</u>	Không	Không	Không	Không
<u>Tĩnh??</u>	Có	Bán	Bán	Không
<u>Rời rạc??</u>				
<u>Đại lý đơn lẻ??</u>				

Loại môi trường

	Solitaire	Backgammon	Mua sắm trên Internet	Taxi
<u>Có thể quan sát được??</u>	Có	Có	Không	Không
<u>Xác định??</u>	Có	Không	Một phần	Không
<u>Tập??</u>	Không	Không	Không	Không
<u>Tĩnh??</u>	Có	Bán	Bán	Không
<u>Rời rạc??</u>	Có	Có	Có	Không
<u>Đại lý đơn lẻ??</u>				

Loại môi trường

	Solitaire	Backgammon	Mua sắm trên Internet	Taxi
<u>Có thể quan sát được??</u>	Có	Có	Không	Không
<u>Xác định??</u>	Có	Không	Một phần	Không
<u>Tập??</u>	Không	Không	Không	Không
<u>Tĩnh??</u>	Có	Bán	Bán	Không
<u>Rời rạc??</u>	Có	Có	Có	Không
<u>Một đại lý??</u>	Có	Không	Có (ngoại trừ đấu giá)	Không

Loại môi trường quyết định phần lớn thiết kế tác nhân

Thế giới thực (tất nhiên) có thể quan sát được một phần, ngẫu nhiên, tuần tự, nang động, liên tục, đa tác nhân

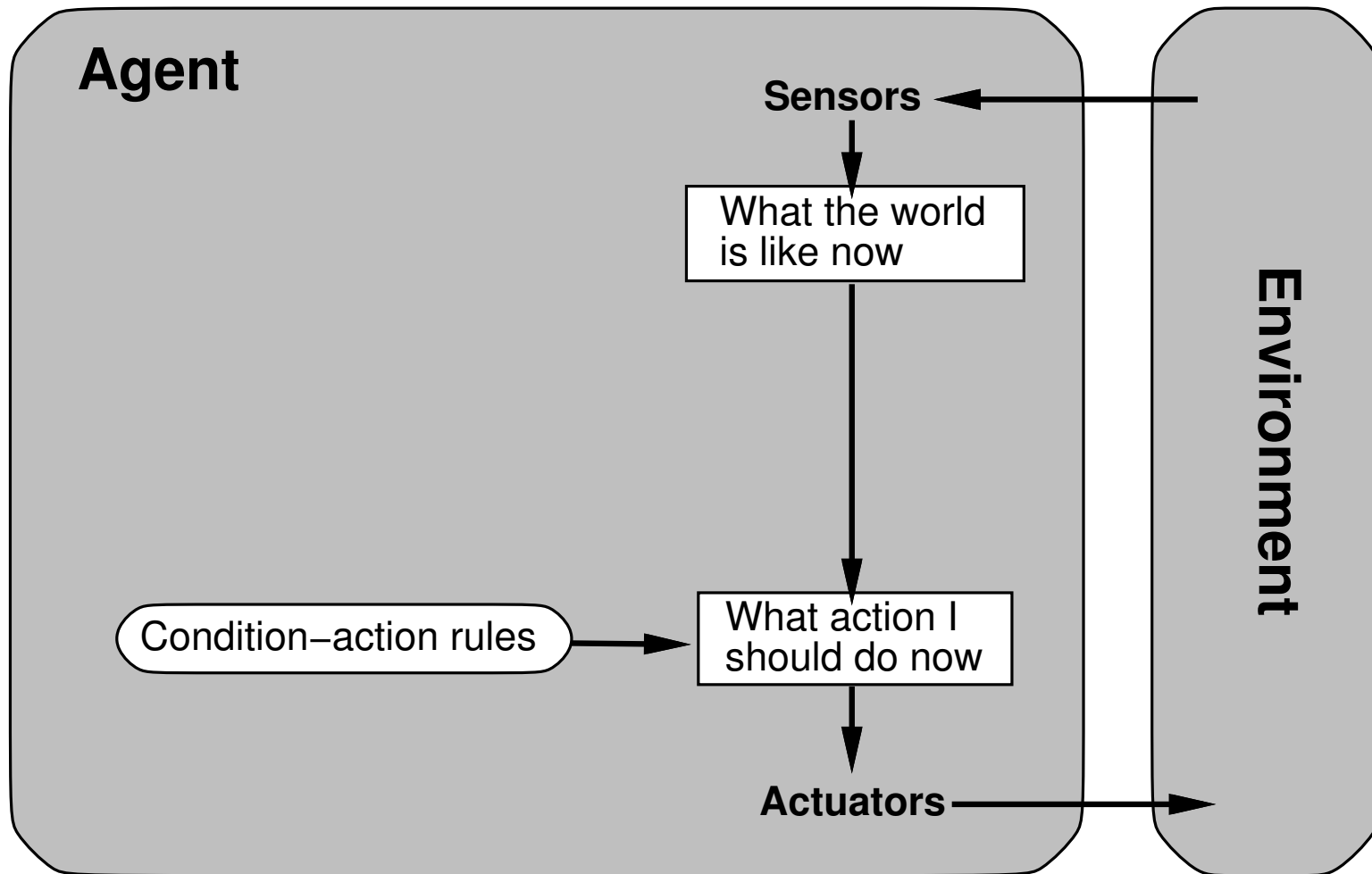
Các loại tác nhân

Bốn loại cơ bản theo thứ tự tổng quát tăng dần:

- tác nhân phản xạ đơn giản
- tác nhân phản xạ có trạng thái
- tác nhân dựa trên mục tiêu
- đại lý dựa trên tiện ích

Tất cả những thứ này có thể được biến thành tác nhân học tập

Các tác nhân phản xạ đơn giản



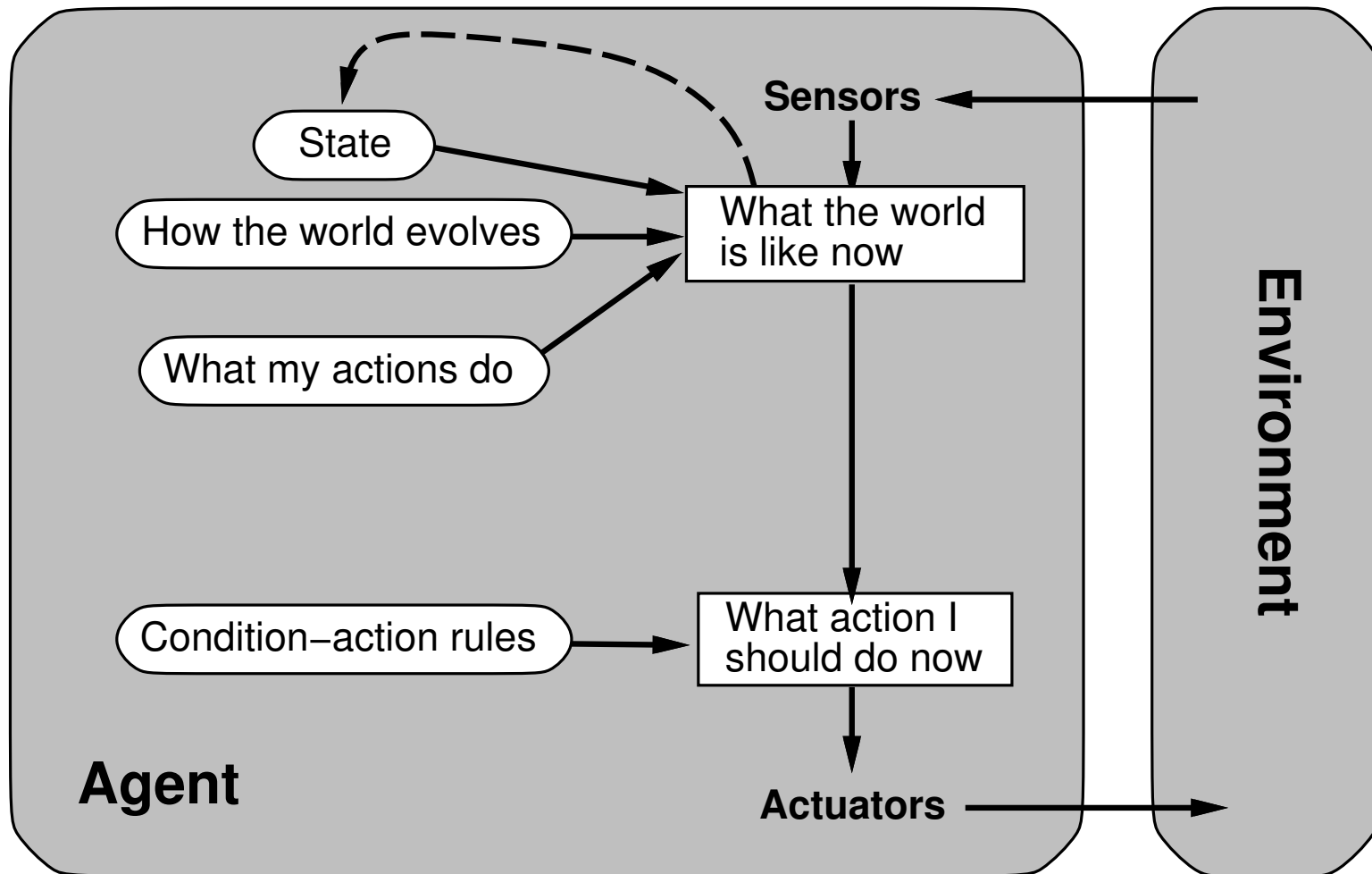
Ví dụ

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

```
(setq joe (make-agent :name 'joe :body (make-agent-body)
                      :program (make-reflex-vacuum-agent-program)))
```

```
(defun make-reflex-vacuum-agent-program ()
  #'(lambda (percept)
      (let ((location (first percept)) (status (second percept)))
        (cond ((eq status 'dirty) 'Suck)
              ((eq location 'A) 'Right)
              ((eq location 'B) 'Left))))))
```

Tác nhân phản xạ có trạng thái

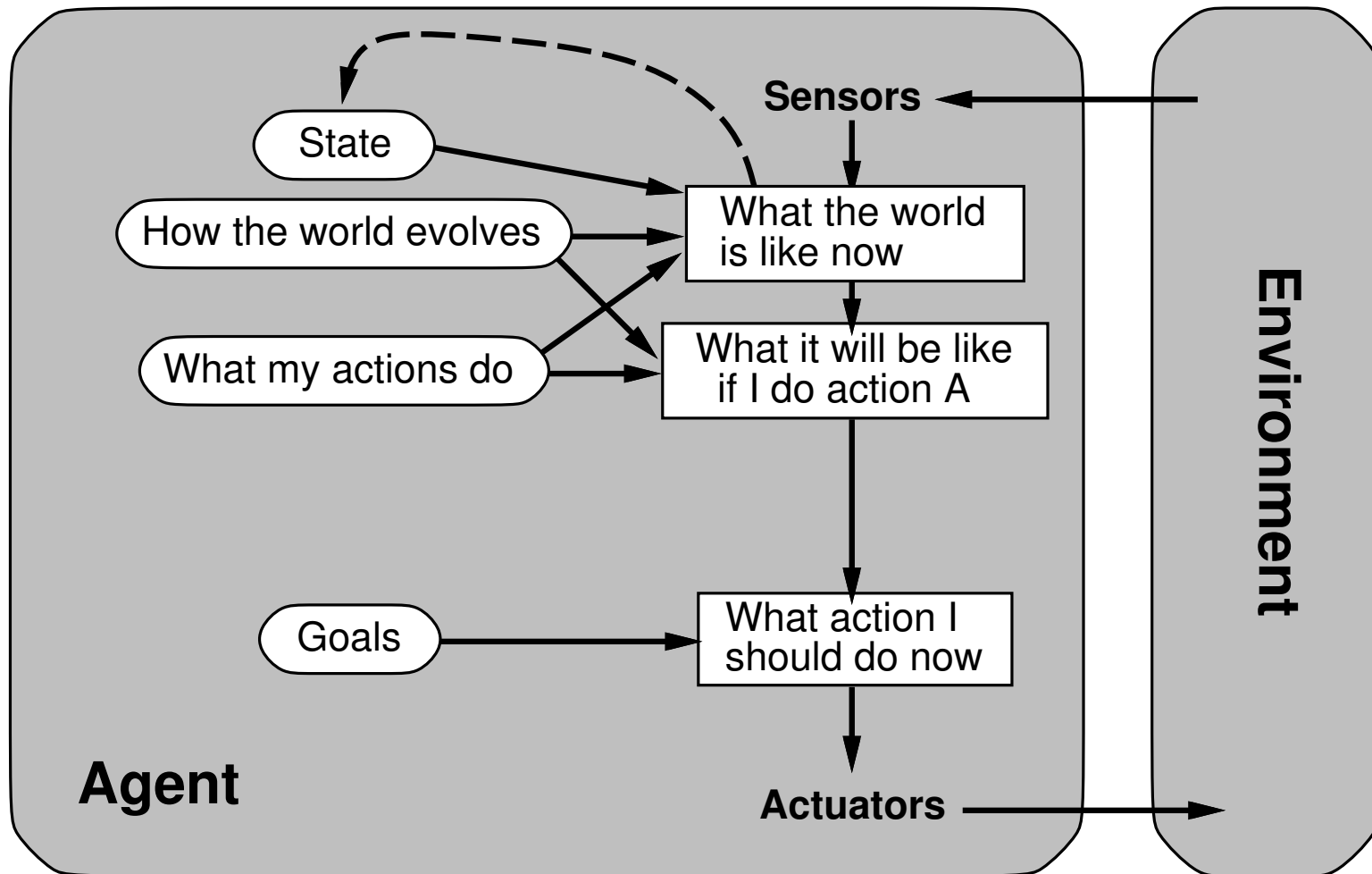


Ví dụ

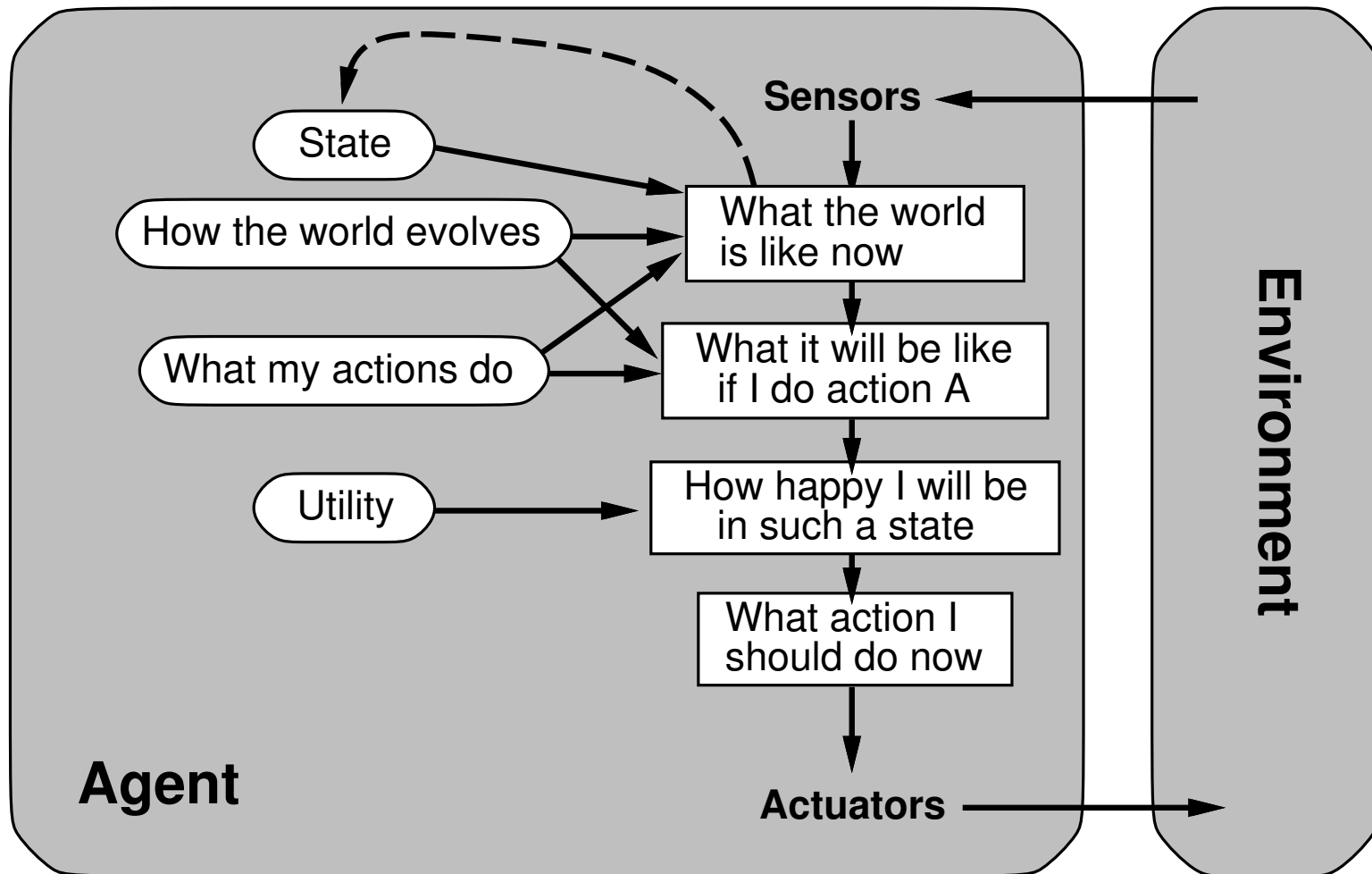
function REFLEX-VACUUM-AGENT(*[location, status]*) **returns** an action
static: *last_A, last_B*, numbers, initially ∞
if *status* = *Dirty* **then** ...

```
(defun make-reflex-vacuum-agent-with-state-program ()  
  (let ((last-A infinity) (last-B infinity))  
    #'(lambda (percept)  
      (let ((location (first percept)) (status (second percept)))  
        (incf last-A) (incf last-B)  
        (cond  
          ((eq status 'dirty)  
           (if (eq location 'A) (setq last-A 0) (setq last-B 0))  
           'Suck)  
          ((eq location 'A) (if (> last-B 3) 'Right 'NoOp))  
          ((eq location 'B) (if (> last-A 3) 'Left 'NoOp)))))))
```

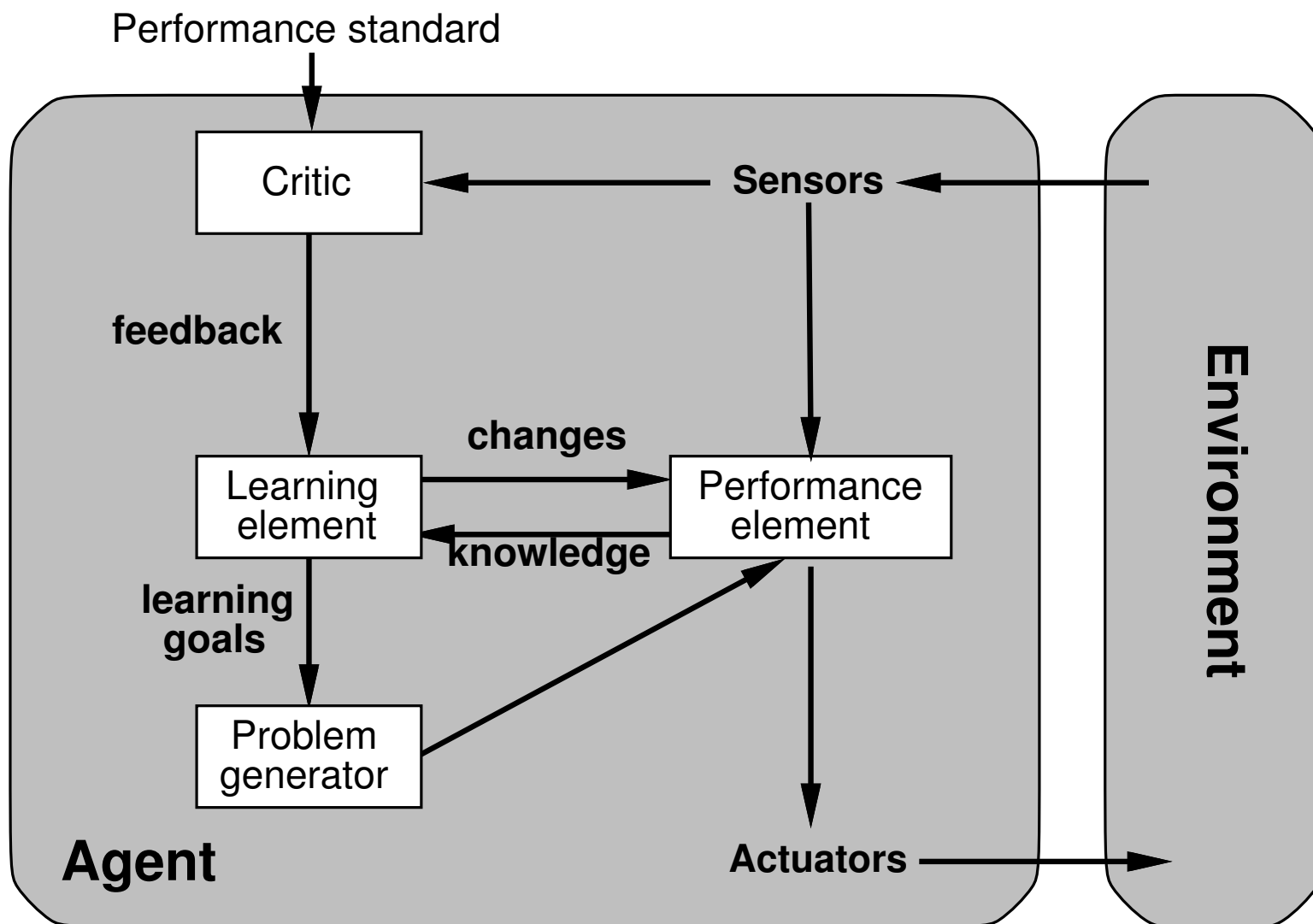

Tác nhân dựa trên mục tiêu



Tác nhân dựa trên tiện ích



Tác nhân học tập



Tóm tắt

Tác nhân tương tác với môi trường thông qua bộ truyền động và cảm biến

Hàm tác nhân mô tả những gì tác nhân thực hiện trong mọi trường hợp

Thước đo hiệu suất đánh giá trình tự môi trường

Tác nhân hoàn toàn hợp lý tối đa hóa hiệu suất mong đợi

Chương trình tác nhân triển khai (một số) chức năng của tác nhân

Mô tả PEAS xác định môi trường tác vụ

Môi trường được phân loại theo nhiều chiều:

có thể quan sát được? xác định? tập? tĩnh? rời rạc? đại lý đơn lẻ?

Một số kiến trúc tác nhân cơ bản tồn tại:

phản xạ, phản xạ với trạng thái, dựa trên mục tiêu, dựa trên tiện ích